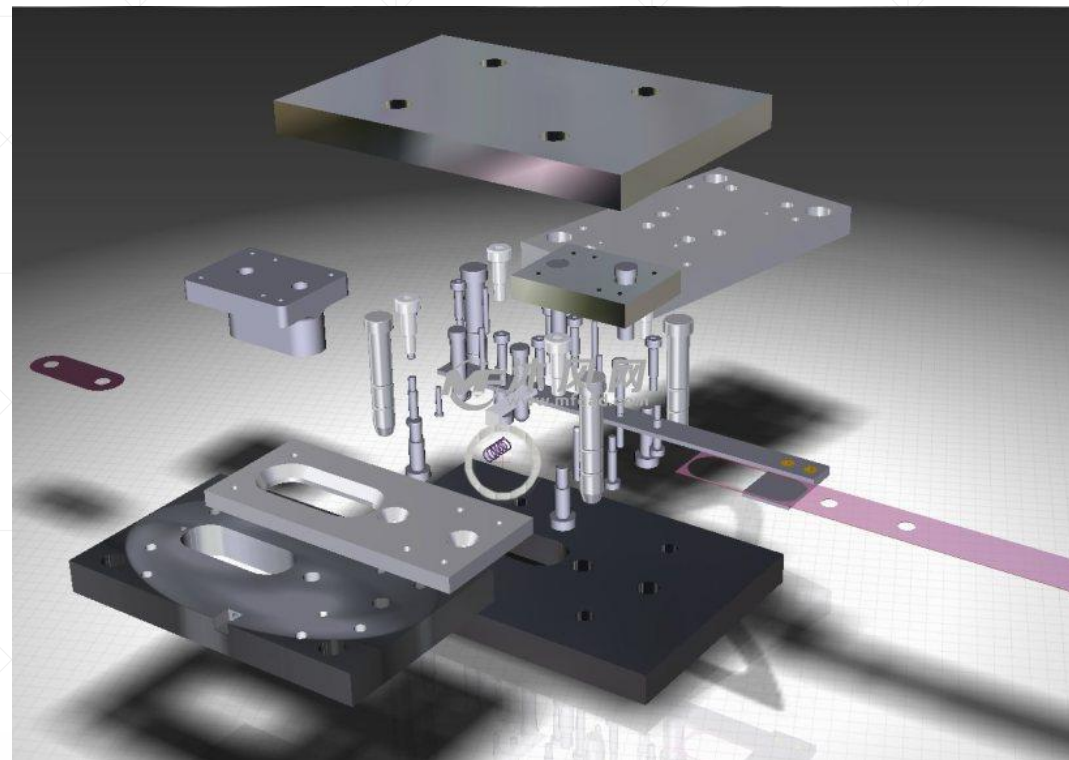


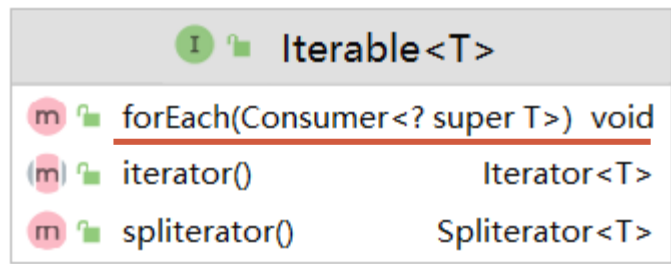


集合的遍历与排序

北京理工大学计算机学院
金旭亮

遍历访问集合对象的新方法

在Java 8中，Iterable<T>接口有了新成员，我们比较感兴趣的是forEach()方法，这个方法比较有用。



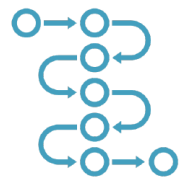
由于Iterable是JDK中许多集合都实现的接口，所以，基于面向对象的多态特性，forEach()方法对于这些集合对象均可用。

使用foreach迭代访问集合示例

```
public class ForEachExample {  
    public static void main(String[] args) {  
        var strings = new ArrayList<String>();  
        strings.add("AAA");  
        strings.add("bbb");  
        strings.add("CCC");  
        strings.add("ddd");  
        strings.add("EEE");  
  
        System.out.println("使用传统方式访问集合");  
        for(String str:strings){  
            System.out.println(str);  
        }  
        System.out.println("使用Lambda表达式访问集合");  
        strings.forEach(str -> {  
            System.out.println(str);  
        });  
  
        System.out.println("更简单的方式，使用方法引用");  
        strings.forEach(System.out::println);  
    }  
}
```

左侧代码中的Lambda表达式，定义了一个实现了Consumer<String>接口的匿名方法（后来被进一步地简化为方法引用），然后被传给了forEach方法。

集合排序



在实际开发中，对集合中的元素进行排序是最常见的一种操作了。



如果集合中的对象实现了Comparable接口，意味着这些对象自己就可以比较大小，JDK中的许多集合可以直接对这样的对象进行排序，比如Collections.sort方法的定义如下：

```
public static <T extends Comparable<? super T>> void sort(List<T> list)
```



如果集合中的对象没有实现Comparable接口，则需要提供一个“第三方”的比较器对象，以便定义两个对象“谁大谁小”的比较规则，这就用到了Comparator<T>接口，sort()方法的另一个重载形式：

```
public static <T> void sort(List<T> list, Comparator<? super T> c)
```

用于比较的Comparator接口

I Comparator<T>		
(m)  <u>compare(T, T)</u>		int
(m)  comparing(Function<? super T, ? extends U>)		Comparator<T>
(m)  comparing(Function<? super T, ? extends U>, Comparator<? super U>)		Comparator<T>
(m)  comparingDouble(ToDoubleFunction<? super T>)		Comparator<T>
(m)  comparingInt(ToIntFunction<? super T>)		Comparator<T>
(m)  comparingLong(ToLongFunction<? super T>)		Comparator<T>
(m)  equals(Object)		boolean
(m)  naturalOrder()		Comparator<T>
(m)  nullsFirst(Comparator<? super T>)		Comparator<T>
(m)  nullsLast(Comparator<? super T>)		Comparator<T>
(m)  reverseOrder()		Comparator<T>
(m)  reversed()		Comparator<T>
(m)  thenComparing(Comparator<? super T>)		Comparator<T>
(m)  thenComparing(Function<? super T, ? extends U>)		Comparator<T>
(m)  thenComparing(Function<? super T, ? extends U>, Comparator<? super U>)		Comparator<T>
(m)  thenComparingDouble(ToDoubleFunction<? super T>)		Comparator<T>
(m)  thenComparingInt(ToIntFunction<? super T>)		Comparator<T>
(m)  thenComparingLong(ToLongFunction<? super T>)		Comparator<T>

如果需要自定义对象的比较规则，则需要用到Comparator接口。

这个接口虽然有N多的成员，但却是一个函数式接口，它定义的抽象方法是compare()。

这个接口定义了一些静态方法，实现了一些常用的比较方法，需要花点时间熟悉一下

对列表进行排序

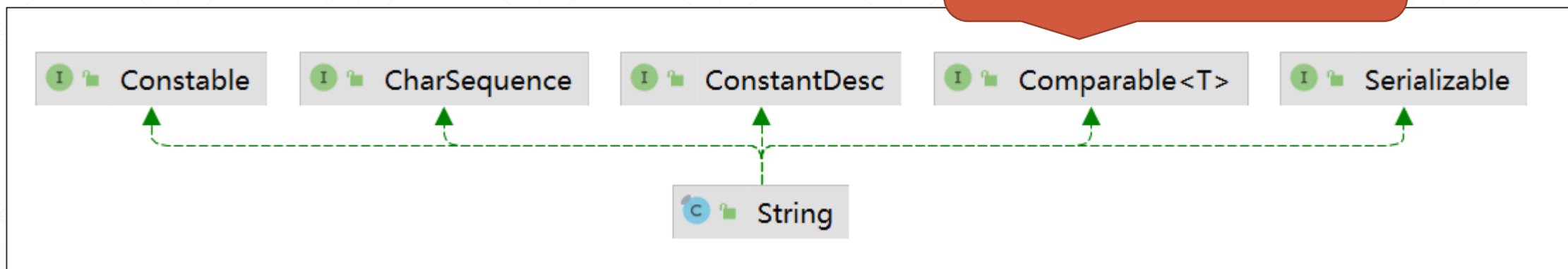
List接口中定义了一个名为sort()的默认方法，用于对集合中的元素进行排序，这意味着所有实现了List接口的集合对象，都可以使用它来进行排序：

```
default void sort(Comparator<? super E> c) {  
    Object[] a = this.toArray();  
    Arrays.sort(a, (Comparator) c);  
    ListIterator<E> i = this.listIterator();  
    for (Object e : a) {  
        i.next();  
        i.set((E) e);  
    }  
}
```



从上述示例源码中可见，List集合的排序，本质上是基于数组排序而实现的。

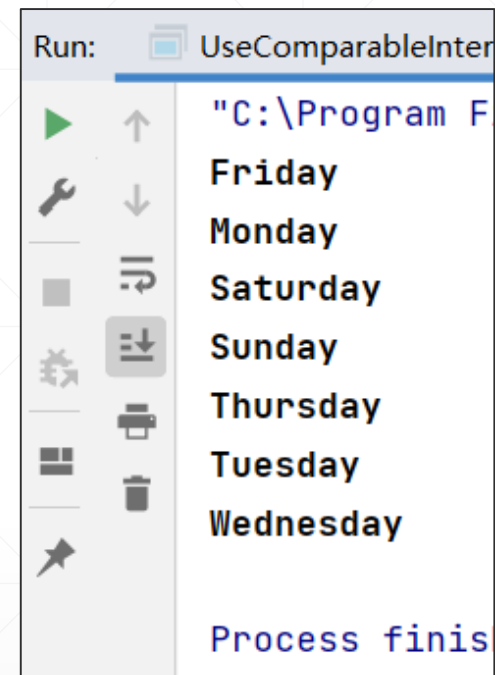
String类型实现了
Comparable接口



由于String类型实现了Comparable接口，所以，字符串对象是可以直接比较大小的。


```
//英文星期单词的集合, 供测试用
static List<String> weekDays = Arrays.asList("Monday", "Tuesday",
    "Wednesday", "Thursday", "Friday", "Saturday", "Sunday");

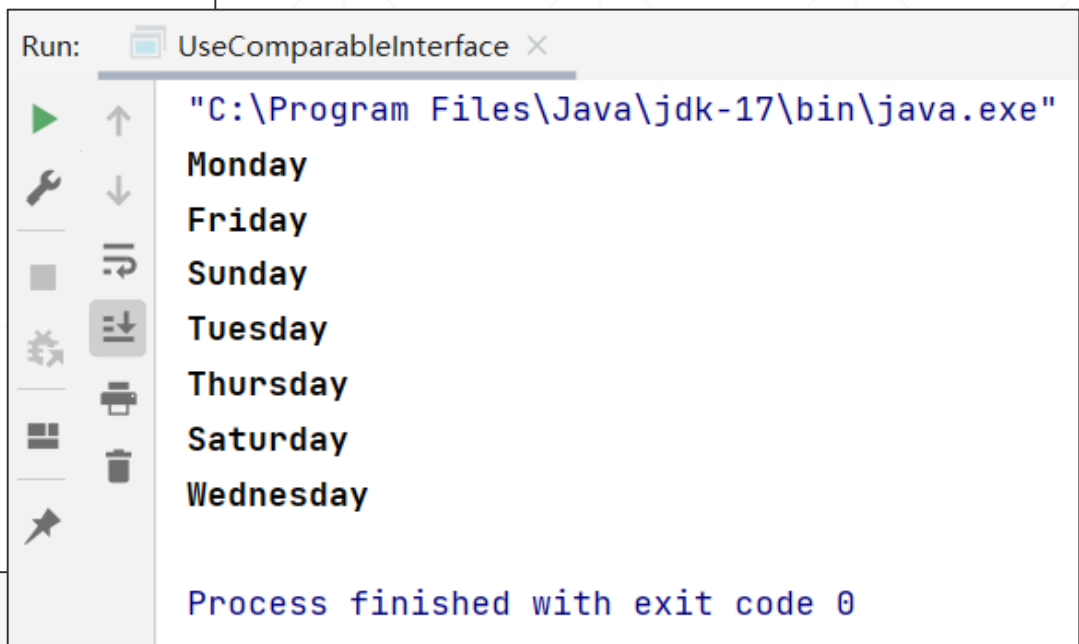
static void sortByItself(){
    //依据字符串自己定义的大小比较规则进行排序
    Collections.sort(weekDays);
    weekDays.forEach(System.out::println);
}
```



当集合中的对象实现了Comparable接口时, 可以直接使用JDK所提供的Collections.sort()方法实现排序。

自定义比较规则的示例

```
static void sortByStringLength() {  
    //定义一个按单词长度比较的比较器  
    Comparator<String> stringLengthComparator = (s1, s2) -> {  
        if (s1.length() > s2.length()) {  
            return 1;  
        } else if (s1.length() < s2.length()) {  
            return -1;  
        } else {  
            return 0;  
        }  
    };  
    weekDays.sort(stringLengthComparator);  
    weekDays.forEach(System.out::println);  
}
```



```
Run: UseComparableInterface x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
Monday  
Friday  
Sunday  
Tuesday  
Thursday  
Saturday  
Wednesday  
  
Process finished with exit code 0
```

代码重构-1

前面那个例子，使用了嵌套的if语句比较两个字符串的长度，其实，Integer类提供了现成的compare()方法完成这个任务，所以，可以简化为以下形式：

```
static void sortByStringLengthUseIntegerCompareMethod() {  
    //定义一个按单词长度比较的比较器,使用Integer类现有的方法实现  
    Comparator<String> stringLengthComparator = (s1, s2) -> {  
        return Integer.compare(s1.length(), s2.length());  
    };  
    weekDays.sort(stringLengthComparator);  
    weekDays.forEach(System.out::println);  
}
```



上述Lambda表达式，主体中只有一个方法调用语句，那么，能不能使用“方法引用”进行简化呢？请试一试！

代码重构-2

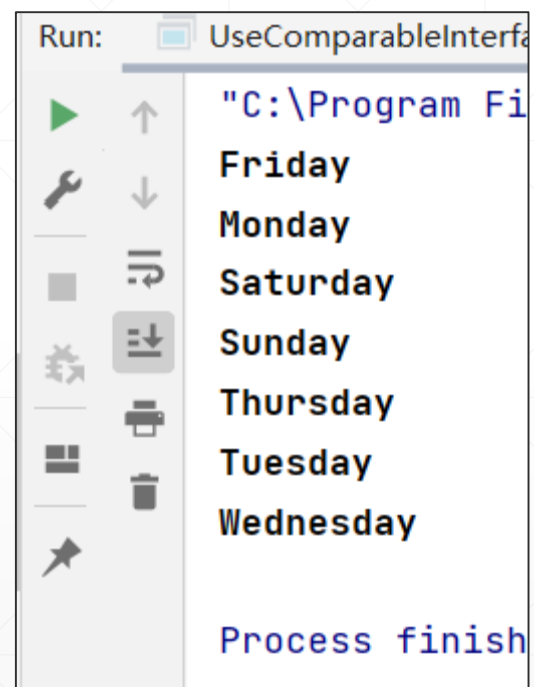
由于比较两个整数的大小是极为常见的场景，所以，Comparator接口干脆提供了一个名为comparingInt()的静态方法做这件事情，于是我们的代码可以进一步地精简：

```
static void sortByStringLengthUseComparatorMethod() {  
    //定义一个按单词长度比较的比较器,使用Integer类现有的方法实现  
    Comparator<String> stringLengthComparator =  
        Comparator.comparingInt(String::length);  
    weekDays.sort(stringLengthComparator);  
    weekDays.forEach(System.out::println);  
}
```

其实，Comparator提供了丰富的方法帮助我们定义各种比较规则，下面我们就来看看它所提供的另外一些有用的方法.....

按自然顺序排序

```
static void sortByNaturalOrder(){  
    //按照自然顺序 (字典序) 排序, 要求对象实现了Comparable接口  
    weekDays.sort(Comparator.naturalOrder());  
    weekDays.forEach(System.out::println);  
}  
  
static void sortByReverseOrder(){  
    //按照自然顺序 (字典序) 反向排序, 要求对象实现了Comparable接口  
    weekDays.sort(Comparator.reverseOrder());  
    weekDays.forEach(System.out::println);  
}
```



comparing方法



当需基于对象的特定属性排序时，Comparator接口中有一个comparing静态方法比较有用：

```
public static <T, U extends Comparable<? super U>> Comparator<T> comparing(  
    Function<? super T, ? extends U> keyExtractor)
```

```
public static <T, U> Comparator<T> comparing(  
    Function<? super T, ? extends U> keyExtractor,  
    Comparator<? super U> keyComparator)
```

这个方法的第一个参数，用于指定使用对象的哪个属性，如果这个属性的类型实现了Comparable接口，则使用第一种形式，否则，可以使用第二种重载形式，使用指定的Comparator完成大小比较工作。

comparing方法示例

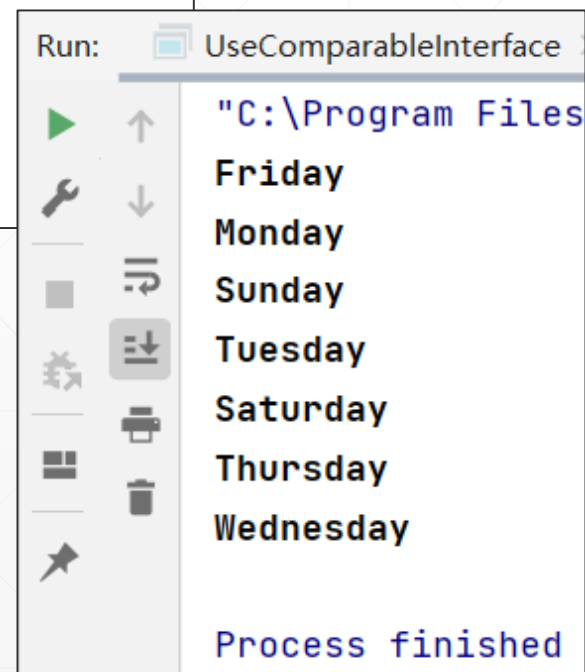
```
static void sortByLowerCase() {  
    //使用comparing()方法, 将字符串转换为小写之后再比较大小  
    weekDays.sort(Comparator.comparing(String::toLowerCase));  
    System.out.println("按小写名称排序");  
    weekDays.forEach(System.out::println);  
}
```

组合排序

Comparator提供了一个thenComparing方法，可以用于组合多个排序规则：

```
static void sortByLinkedComparator() {  
    //按单词长度排序，如果长度相同则按字典序排序  
    Comparator<String> linkedComparator = Comparator.comparingInt(String::length)  
        .thenComparing(Comparator.naturalOrder());  
    weekDays.sort(linkedComparator);  
    weekDays.forEach(System.out::println);  
}
```

掌握了这些介绍的知识，你应该已经知道如何对集合中的元素进行排序了，现在，请使用已经掌握的知识，完成后面布置的练习.....



练习



现有一个学生对象集合，每个学生都有Java考试成绩和学号两个信息。

请编写代码，按照Java考试成绩逆序排列对象集合元素，如果两学生的成绩一样，则依据其学号进行排序。